Formalisme architectural de FulleGenBrain_3

Architecture compositionnelle, mémoire vivante, recherche maîtrisée et validation empirique

Spécification intégrée – évolution de FulleGenBrain_3

25 octobre 2026

Abstract

FulleGenBrain_3 est spécifié comme un système cognitif dont le principe structurel central est la programmation compositionnelle. Les compétences, méthodes, règles, opérateurs, modèles du monde et procédures de validation sont décrits comme des objets typés, versionnés, composables et révisables. L'architecture articule observation multi-vues, empreintes DDD, maillage MG/MGR/MGR2/ASCR, synthèse de programmes et de règles, reproduction/clonage, recherche active, raisonnement spécialisé, sémantique, mémoire de provenance, contrôle autonome et vérification indépendante. Le système n'intègre et ne dépend d'aucun modèle de langage neuronal, y compris eLLM. Les techniques de représentation multi-résolution, de prédiction, de calibration ou de génération de candidats ne sont reprises que sous forme de mécanismes indépendants, explicites et évaluables. Le document distingue les contrats théoriques, les preuves conditionnelles, les sources historiques, les tests locaux et les capacités qui requièrent encore une implémentation intégrée. Il ne préjuge ni le résultat de la construction ni les performances futures.

Contents

1 Conventions et statut des affirmations 3

2 Objectif et principes non négociables 3

3 Architecture d'ensemble : un organisme à quatre plans coordonnés 4

4 Contrat universel des objets et statuts 5

5 Programmation compositionnelle : noyau central de FulleGenBrain_3 5

6 Ordonnancement compositionnel et lutte multicouche contre l'explosion combinatoire 7

7 Registre de compétences et architecture stratégique 8

8 Observation, empreintes DDD et sondes actives 9

9 Maillage relationnel : MG, MGR, MGR2, ASCR et extensions 9

10 Induction de règles, programmes et modèles 10

11 Observation, reproduction et clonage comportemental 11

12 Modèles de transition et monde simulé : usage conditionnel, testable et composable 11

13 Langage et sémantique sans modèle de langage neuronal 13

14 Raisonnement formel et place d'eGenBrain v4 14

15 Mémoire, provenance et révision continue 14

16 Mémoire vivante : activation, compression et circulation d'information 15

17 Contrôleur, objectifs et allocation de ressources 16

18 Résilience opérationnelle et continuité 16

19 Sécurité des effets sans limitation artificielle des tâches 17

20 Preuve de composition et portes de vérification 17

21 Intégration complète de la théorie compositionnelle dans FulleGenBrain_3 18

22 Protocole d'évaluation de FulleGenBrain_3 20

23 Plan d'intégration recommandé 21

24 Correspondance avec FulleGenBrain_1, AAM O94.6 et eGenBrain v4 22

25 Questions ouvertes comme tâches de construction 22

26 Conclusion et statut 22

1 Conventions et statut des affirmations

1.1 Version et mandat

Ce document remplace la spécification architecturale FulleGenBrain_1 par une version qui recentre toutes les capacités autour d'un langage de composition explicite. La théorie autonome de la programmation compositionnelle est livrée dans le même dossier et constitue le fondement théorique de la présente architecture. Les identifiants de composants, contrats et transitions ci-dessous sont des contrats cibles, pas une affirmation que toutes les parties ont déjà été implémentées.

Les marqueurs de statut sont courts et distincts :

- **SOURCE** : constat présent dans une source examinée;
- **SPÉCIFICATION** : contrat défini par ce document;
- **PREUVE SOUS HYPOTHÈSES** : conclusion mathématique dérivée d'hypothèses explicites;
- **TEST LOCAL** : vérification sur un cas borné;
- **NON INTÉGRÉ** : chaîne complète non exécutée dans cette livraison;
- **À ÉVALUER** : question demandant une mesure sans présumer son résultat.

1.2 Règles d'interprétation

Un exemple qui fonctionne n'établit pas une loi universelle. Un compilateur qui réussit à construire un artefact ne prouve pas que sa sémantique métier est correcte. Une dérivation valide dépend de ses prémisses. Une bonne imitation n'est pas à elle seule une reconstruction de mécanisme. Un score heuristique n'est pas une certification. Toute sortie conserve son type, ses hypothèses, ses preuves, ses versions, son périmètre de validité et le statut qui lui est effectivement justifié.

2 Objectif et principes non négociables

FulleGenBrain_3 doit pouvoir observer, représenter, organiser, composer, tester, apprendre, expliquer, reproduire, prédire, raisonner, vérifier, réviser et transformer ses propres programmes de résolution. Il ne suffit pas d'avoir les modules : il faut que leur coopération soit définie par des contrats de flux, de contrôle, de preuve, de coût et de récupération.

1. **Composition comme colonne vertébrale.** Toute capacité complexe est soit un composant typé, soit une composition de composants, soit un mécanisme du runtime qui vérifie et gouverne ces compositions.
2. **Aucune dépendance à un modèle de langage neuronal.** Aucun LLM, eLLM, poids de modèle de langage externe ou service de génération neuronal n'est un composant d'exécution, un fournisseur de connaissances implicite, une voie de repli ou une dépendance obligatoire.
3. **Distinction des rôles épistémiques.** L'observation, l'hypothèse, la règle candidate, la dérivation, le résultat testé et le résultat certifié sont des statuts différents.
4. **Pluralité avant fermeture prématurée.** Lorsque les données n'identifient pas un modèle unique, les alternatives et leurs désaccords sont conservés; le contrôleur peut demander une observation discriminante.
5. **Évolution tracée et réversible.** Une nouvelle compétence, une transformation ou une optimisation est un candidat soumis à typage, test, preuve disponible, évaluation et retour arrière.
6. **Résilience explicite.** Panne, timeout, manque de données, corruption, conflit et incertitude possèdent des états et stratégies propres. Aucune dégradation n'est masquée sous un statut de succès.

7. **Pas de tâche abandonnée par défaut.** Si une fonction nécessaire ne possède pas encore de mécanisme suffisant, elle génère une obligation de formalisation, de construction et d'évaluation. Le contrôleur peut exposer un état de travail incomplet sans requalifier l'objectif en impossible.

3 Architecture d'ensemble : un organisme à quatre plans coordonnés

3.1 Plan structurel : catalogue et compositions

Le catalogue contient des compétences atomiques, des méthodes composées, des règles/invariants et des modèles versionnés. Chaque composant possède un contrat d'entrée/sortie, des effets, des ressources, des préconditions, des postconditions, des tests, une provenance et une politique de révocation. Les compositions sont des AST ou graphes typés qui sélectionnent les composants, raccordent leurs valeurs, organisent leurs branches, définissent les récupérations et décrivent le flux de preuves.

3.2 Plan dynamique : runtime et contrôleur

Le runtime exécute les compositions approuvées; le contrôleur choisit les objectifs, budgets, probes, chemins alternatifs et conditions d'arrêt; l'ordonnanceur gère la séquence et le parallélisme lorsque les effets l'autorisent. Le contrôleur ne peut pas transformer une proposition en résultat certifié. Le runtime n'invente pas de compétences absentes du registre. Le registre, le synthétiseur et le vérificateur restent des responsabilités distinctes.

3.3 Plan évolutif : découverte et révision

Le système propose de nouveaux blocs, règles, programmes, modèles de transition et compositions à partir d'observations, de contre-exemples, d'analogies calculables, de recherche grammaticale et de tests. Le candidat est exécuté en environnement d'essai; la validation compare la version candidate à une référence, à des données tenues à l'écart et à une suite de régression. L'activation est versionnée. Une régression ouvre une transaction de retour à la dernière version approuvée et préserve les preuves de l'épisode.

3.4 Plan mémoire-ressources : activation et continuité

La mémoire vivante et le gestionnaire de ressources relient les autres plans sans leur substituer. La mémoire définit quels objets sont actifs, dormants, archivés, périmés ou révoqués; l'ordonnanceur arbitre le front de synthèse, les réveils et les vérifications; le magasin durable garde les payloads et checkpoints avec provenance; les vagues d'informations transportent des deltas typés vers des abonnés. Les événements de révocation et de changement de version ont priorité sur les événements exploratoires.

3.5 Cycle de vie global

$$\begin{aligned}\text{Objectif} &\to \text{Composer une voie} \to \text{Observer/Calculer} \\ &\to \text{Comparer les modèles} \to \text{Vérifier} \to \text{Mémoriser/Réviser}.\end{aligned}$$

Ce cycle n'est pas un pipeline figé. Une observation inattendue peut relancer la définition de tâche; un contre-exemple peut changer un modèle; une panne peut déclencher un composant de remplacement; une vérification invalide peut révoquer les descendants de la preuve concernée. Toute composition suit un cycle de vie versionné – candidate, bien typée, évaluée, approuvée dans un domaine, active, idle, sleeping, réveillée pour revalidation, stale ou révoquée. Chaque transition enregistre sa cause, son coût, ses entrées, sa sortie et son statut.

4 Contrat universel des objets et statuts

4.1 Objet commun

Tout élément manipulable est encapsulé dans

$$o = (id, kind, payload, type, context, time, source, status, scope, version, \\ deps, uncertainty, certificate).$$

Le champ `kind` distingue donnée brute, observation, token/symbole, parse, référence, fait, règle, candidat, programme, composition, état, action, événement, trajectoire, prédiction, résultat, contre-exemple, probe, certificat et entrée mémoire. Le champ `source` décrit la provenance; `scope` précise le domaine où l'élément s'applique; `deps` est un ensemble de dépendances avec relations typées; `uncertainty` décrit les ambiguïtés restantes. Un type non encore reconnu doit être conservé comme donnée opaque versionnée, puis donné lieu à un adaptateur formalisé.

4.2 États épistémiques

Statut	Signification opérationnelle
RAW	donnée reçue, contenu non encore interprété
OBSERVED	donnée observée sous un protocole précisé avec provenance
PARSED	structure calculée par un parseur déclaré; pas une garantie de vérité sémantique
HYPOTHESIS	interprétation ou mécanisme candidat
RULE-CANDIDATE	règle reifiée avec portée et conditions de test
PROGRAM-CANDIDATE	programme ou composition de résolution proposée
SUPPORTED	compatible avec un ensemble explicite de preuves et un domaine précis
DERIVED	conséquence d'une règle ou d'un opérateur nommé et de prémisses tracées
TESTED	évalué sur un protocole/test identifié avec résultats enregistrés
CERTIFIED	accepté par un vérificateur défini dans un périmètre défini
REFUTED	contredit par un contre-exemple dans les conditions indiquées
UNRESOLVED	plusieurs candidats ou états demeurent compatibles
STALE	dépendance, contexte, source ou version ont changé; réévaluation obligatoire
WITHDRAWN	retrait explicite; interdiction d'usage comme prémisse active non revalidée
PARTIAL	résultat utile mais incomplet, avec les obligations non satisfaites
FAILED	exécution échouée avec catégorie, trace et récupération éventuelle

Les promotions et rétrogradations sont des transactions journalisées. Une hypothèse n'est pas promue par son score seul; un candidat testé demeure lié au protocole; un certificat est limité aux hypothèses, version de vérificateur et domaine notés sur celui-ci.

5 Programmation compositionnelle : noyau central de FulleGenBrain_3

5.1 Contrat d'un composant

Chaque bloc c suit le contrat

$$c = (id, version, A, B, Pre, Post, Effects, Resources, TracePolicy, Verifier, Recovery, Tests),$$

où A, B sont les types d'entrée/sortie, $Pre/Post$ les contrats logiques, $Effects$ le système d'effets, $Resources$ le budget, $TracePolicy$ les événements conservés, $Verifier$ le contrôle indépendant pertinent et $Recovery$ la stratégie de reprise/compensation. Les composants peuvent être purs, étatful, observationnels, transactionnels ou interactifs; ces classes sont explicites. Le contrat distingue ce qui est spécifié, testé, prouvé et mesuré.

5.2 Grammaire conceptuelle du DSL

La grammaire exécutable évolue par versions; la base architecturale comprend les opérateurs suivants :

$$
\begin{aligned}
p ::= {} & \mathsf{Id} \mid \mathsf{Call}(c, e) \mid \mathsf{Seq}(p_1, \ldots, p_n) \mid \mathsf{Par}(p_1, \ldots, p_n) \\
& \mid \mathsf{If}(g, p_t, p_f) \mid \mathsf{Map}(p, e) \mid \mathsf{Fold}(p, e, a) \mid \mathsf{Loop}(g, p, I) \mid \mathsf{Choose}(p_1, \ldots, p_n) \\
& \mid \mathsf{Select}(\pi, p_1, \ldots, p_n) \mid \mathsf{Compose}(m, \theta) \mid \mathsf{Probe}(q) \mid \mathsf{Observe}(q) \\
& \mid \mathsf{Verify}(p, v) \mid \mathsf{Recover}(p, r) \mid \mathsf{Checkpoint}(p, k) \mid \mathsf{Txn}(p, \kappa) \\
& \mid \mathsf{Revoke}(d) \mid \mathsf{Synthesize}(G, D) \mid \mathsf{Transform}(p, \tau) \\
& \mid \mathsf{Fix}(x, p) \mid \mathsf{Hibernate}(c) \mid \mathsf{Wake}(c, q) \mid \mathsf{Compress}(m, k) \\
& \mid \mathsf{Summarize}(m, k, \epsilon) \mid \mathsf{Restore}(k) \mid \mathsf{EmitWave}(w) \\
& \mid \mathsf{ReceiveWave}(w) \mid \mathsf{ResumeSearch}(k).
\end{aligned}
$$

La grammaire n'est pas fermée à jamais : un nouveau cas d'utilisation requiert la spécification formelle de son opérateur, l'analyse de ses effets, une sémantique, une stratégie de validation et des tests de non-régression. La compatibilité est versionnée; un runtime qui ne comprend pas un nœud ne doit pas l'ignorer silencieusement.

5.3 Lois de base de composition

La séquence $g \circ f$ est admissible si la sortie de f satisfait le type et la précondition de g. L'associativité est garantie pour les compositions pures selon l'égalité des fonctions; pour les programmes avec effets, elle n'est autorisée que lorsque les traces, transactions, exceptions et effets observables restent équivalents. L'identité ne supprime aucun événement exigé par le contrat. Le parallélisme requiert une analyse de lectures/écritures, d'effets externes et de ressources. Une transformation telle que fusion, permutation, factorisation ou mise en cache est un candidat avec une obligation d'équivalence, pas une optimisation présumée sûre.

5.4 Synthèse comme capacité de premier rang

Le synthétiseur reçoit l'objet objectif G, un ensemble d'exemples, une grammaire de candidats, des coûts, des contraintes et un oracle indépendant. L'espace de synthèse est le sous-ensemble des compositions dont types, effets, budgets, risque et niveau de preuve sont compatibles avec G. Il génère plusieurs compositions, les normalise, contrôle types et effets, exécute les cas observés, recherche des contre-exemples et teste les compositions inédites. Il garde les alternatives compatibles. Le classement peut privilégier coût, fidélité, robustesse ou information gagnée, mais aucun score heuristique n'a le droit de certifier la solution.

5.5 Apprentissage des blocs

Lorsqu'une séquence d'actions réussit sur plusieurs exemples, le système peut proposer de la factoriser comme sous-composition nommée. Cette induction comprend : comparaison des traces; alignement des entrées/sorties; séparation des invariants et des paramètres; construction d'une abstraction; tests sur variantes; contrôle des effets; mesure de couverture; validation du contrat; ajout au catalogue comme candidat; approbation ou rejet motivé. Une macro apprise ne devient pas une primitive réputée universelle sans les tests requis.

5.6 Réutilisation à plusieurs niveaux

La composition s'applique à tous les niveaux : opérations élémentaires; méthodes de domaine; stratégies; workflows; contrôleurs; procédures d'apprentissage; protocoles de vérification; plans de récupération; transformations du modèle lui-même. Une compétence peut être répétée dans plusieurs voies si cela augmente la fiabilité ou donne un contrôle indépendant, à condition que les rôles et les responsabilités soient différents ou que l'indépendance recherchée soit mesurée. La duplication non justifiée des calculs est évitée par cache traçable, pas par suppression aveugle.

6 Ordonnancement compositionnel et lutte multicouche contre l'explosion combinatoire

6.1 Principe : la veille réduit le working set, pas l'espace logique

La croissance de l'espace de synthèse demeure exponentielle dans le pire cas. La mise en veille est retenue parce qu'elle réduit la résidence en mémoire et permet de garder un grand catalogue sans garder simultanément tous les objets actifs; elle ne remplace pas la stratégie de génération et ne diminue pas automatiquement le nombre de candidats. FulleGenBrain_3 associe donc veille/réveil, génération paresseuse, filtrage par types et contrats, recherche best-first reprenable, partage de DAG, normalisation, contre-exemples et budgets journalisés.

6.2 Frontière de recherche et signature canonique

Chaque candidat est un objet $q = (AST, \sigma, depth, cost, priority, deps, proofState, trace)$. Sa signature σ est calculée sur un AST canonique versionné. La signature ne fusionne que les structures identiques selon cette normalisation; une réduction d'équivalence plus agressive exige un certificat d'équivalence dans le domaine choisi. Les candidats qui violent un type, un contrat, une unité, une capacité ou une contrainte de ressource sont rejetés avant exécution avec un motif traçable. Le score d'ordre ne certifie rien.

6.3 Politique de recherche

Le moteur utilise, selon l'objectif, recherche en largeur, approfondissement itératif, best-first, branch-and-bound, synthèse contre-exemple-guidée ou combinaison de ces techniques. La sélection expose une fonction d'ordre et un budget; le front non traité et l'état des déduplications sont persistés. L'ordonnanceur réserve une part du budget au contrôle des preuves, à la propagation de révocations et aux tâches vieillissantes afin qu'une file de recherche lucrative ne puisse pas affamer les fonctions de sûreté ou la maintenance mémoire.

6.4 Règle de veille et interdictions d'éviction

Une composition inactive peut être mise en sommeil si elle n'a aucun bail d'exécution, n'est pas épinglée, ne participe pas à une transaction non close et n'est pas nécessaire à la propagation d'une révocation. Avant le sommeil, le runtime écrit atomiquement payload canonique, hash, version, dépendances, état de recherche, contrats et pointeurs de preuve. Au réveil, il vérifie l'intégrité, les versions des dépendances, le contexte et les capacités. Une composition stale passe par revalidation ou reconstruction; il n'y a pas de réveil automatique vers un état approuvé. Un coût de réveil excessif peut être une raison de garder un objet résident, décidée par une mesure de workload.

6.5 Politique d'expansion adaptative

La priorité peut combiner $priority(q) = \alpha I(q) + \beta V(q) - \gamma C(q) - \eta R(q)$, où I est l'information attendue, V la valeur estimée des contraintes satisfaites, C le coût et R le risque ou la fragilité. Les termes, normalisations, coefficients et versions sont enregistrés. Les coefficients sont appris ou adaptés seulement à partir de résultats évalués; les décisions critiques peuvent demander une

politique déterministe et bornée. La recherche conserve un mode de poursuite de couverture pour ne pas enfermer le système dans ses propres préférences historiques.

6.6 Sorties de budget et continuité

Les statuts incluent BUDGET-AVAILABLE, BUDGET-EXHAUSTED-RESUMABLE, FRONTIER-EMPTY, CANDIDATE-FOUND et CANDIDATE-VERIFIED. Un front vide ne veut dire absence de solution que par rapport à la grammaire, aux règles d'élagage et au domaine parcourus. Toute augmentation ultérieure de grammaire invalide cette conclusion locale. La reprise à budget supérieur conserve les signatures déjà vues mais réouvre les familles pertinentes lorsque la représentation ou les contraintes ont évolué.

7 Registre de compétences et architecture stratégique

7.1 Familles de blocs

Famille	Responsabilité	Exemples de blocs
Observation	capturer des entrées accessibles et établir leur provenance	observation fichier/flux/trace; parseur de données; capture d'épisode; sonde
Représentation	construire les vues et structures typées	tokenisation déterministe; analyse syntaxique; DDD; normalisation; types et unités
Maillage	relier faits, règles, dépendances, candidats et contextes	MG; MGR/MGR2; ASCR; graphe temporel; provenance
Synthèse	proposer et raffiner des règles, programmes et compositions	recherche grammaticale; anti-unification; synthèse d'AST; apprentissage de sous-programmes
Exécution	appliquer des règles, calculs, outils et compositions	eGenBrain v4 adapté; solveurs; moteur de contraintes; runtime compositionnel
Expérimentation	sélectionner les observations utiles et contraster des hypothèses	choix de probe; plan d'essais; mutation de cas; générateur de contre-exemples
Monde et trajectoires	prédire et simuler les transitions d'état	modèle de transition; planification; recherche de trajectoire; simulateur
Vérification	contrôler indépendamment le résultat et les preuves	vérificateur de contrat; vérifieur de preuve; contrôle d'unités; oracle de test externe
Mémoire	préserver données, programmes, preuves et dépendances	mémoire persistante; cache de résultats; checkpoints; registre de versions
Résilience	diagnostiquer et reprendre après panne	retry policy; checkpoint/restart; compensation; gestion de circuit; rollback
Contrôle	choisir le prochain objectif et les budgets	contrôleur de tâche; ordonnanceur; gestion de conflits; politique d'abstention

Aucune famille ne remplace toutes les autres. La composition est l'organe d'intégration, le

contrôleur l'organe de régulation, la mémoire l'organe de continuité, le vérificateur la porte de justification, et l'observation le lien avec les données réelles.

8 Observation, empreintes DDD et sondes actives

8.1 Observations comme mesures

Soit x une situation et q une sonde dans un contexte c à une résolution r. La valeur observée est $O(q, x, c, r)$ et l'empreinte

$$\phi_{Q,C,R}(x) = \big(O(q_1, x, c_1, r_1), \ldots, O(q_m, x, c_m, r_m)\big).$$

L'empreinte est relative au jeu de sondes, au contexte et aux résolutions. Deux situations égales sous cette empreinte sont indiscernables pour ce protocole, non nécessairement identiques dans le monde. La révision de Q, C, R modifie la partition observée et nécessite versionnement.

8.2 Multi-vues et abstractions hiérarchiques

Une observation peut être projetée dans plusieurs vues complémentaires : octets, champs structurés, tokens/symboles, syntaxe, références, sens, événements, trajectoire ou contraintes numériques. Les vues gardent une référence à la source et la transformation qui les a produites. Une vue plus abstraite doit préserver les distinctions que ses tâches aval exigent; si elle les efface, le système doit revenir à une vue plus fine ou acquérir une nouvelle observation.

8.3 Choix de probes

Le contrôleur maintient un ensemble de candidats $H = \{h_i\}$. Une probe est utile si ses résultats prévus séparent des candidats actuellement compatibles. La sélection peut optimiser un gain d'information estimé, le coût d'acquisition, le risque, la capacité de discrimination et la dépendance envers les modèles. Le score est un guide de recherche. La preuve de distinction découle ensuite de la sémantique de la probe et du résultat observé, lorsque les hypothèses sont suffisantes.

8.4 Garde-fous d'observation

Les journaux séparent observation directe, dérivation, simulation et contrefactuel. Toute observation reste dans le périmètre d'accès déclaré. Une donnée inaccessible n'est jamais enregistrée comme capturée. Si une capacité manque, le système spécifie et construit un adaptateur autorisé, ou marque explicitement le champ comme non observé.

9 Maillage relationnel : MG, MGR, MGR2, ASCR et extensions

9.1 MG : mémoire persistante

MG conserve les objets durablement réutilisables : faits observés, règles candidates et acceptées, opérateurs, programmes, compositions, certificats, contre-exemples, contextes, transitions, versions et liens de provenance. Chaque entrée indique sa portée et les conditions de révocation. Les éléments temporaires n'y sont promus qu'après validation selon une politique explicitée.

9.2 MGR/MGR2 : maillages de travail

MGR construit un contexte relationnel local depuis une requête, une situation ou une recherche; MGR2 affine ou restructure ce contexte après les résultats intermédiaires. Ils restent transitoires et versionnés par tâche. Une hypothèse temporaire ne modifie pas la mémoire persistante par effet de bord. La promotion vers MG est une transaction contrôlée.

9.3 ASCR : sélection des candidats et témoins

ASCR gère la sélection de candidats, leur provenance, leurs critères d'inclusion et les éléments qui permettent de les contredire. Le score de classement reste séparé de l'état épistémique. Le pool conserve les candidats concurrents utiles et les résultats incompatibles. Le choix d'un candidat pour la prochaine expérience ne le transforme pas en candidat vrai.

9.4 Nouveaux maillages fonctionnels

FulleGenBrain_3 distingue au moins quatre vues reliées : (i) le graphe de provenance et de dépendances; (ii) le graphe d'hypothèses, objections et contre-exemples; (iii) le graphe temporel/dynamique des états, événements et transitions; (iv) le graphe de composition des programmes et compétences. Ces vues partagent des identifiants typés mais gardent des relations sémantiquement distinctes. Elles peuvent être matériellement réunies dans un store relationnel commun si les contrats de requête les distinguent.

9.5 Fermeture, invalidation et calcul dérivé

Chaque dérivation enregistre la liste exacte des prémisses, des règles et des opérateurs. Si un fait, une règle, un opérateur ou un contexte est retiré ou modifié, le graphe de dépendance calcule tous les descendants à réévaluer. Les cycles de dépendance sont groupés en composantes fortement connexes. Une sortie dépendant d'une prémisse retirée devient STALE puis WITHDRAWN ou est reconstruite avec une preuve indépendante. Une entrée mémoire ne peut pas demeurer CERTIFIED simplement parce qu'elle était autrefois valide.

10 Induction de règles, programmes et modèles

10.1 Espace général des candidats

Le candidat général est

$$h = (kind, syntax, contract, scope, assumptions, evidence, predictions, cost, status, trace),$$

où `kind` peut être règle, opération, programme, composition, parse, interprétation sémantique, modèle de transition ou politique. La grammaire définit les formes actuellement recherchées; elle est versionnée et peut être étendue. Le système ne doit pas limiter par principe l'induction à la découverte de règles logiques.

10.2 Cycle de découverte

1. construire des exemples observés et séparer données d'apprentissage, de validation et de test;
2. produire plusieurs candidats à partir de recherche syntaxique, induction, analogies structurées, modèles d'exécution et sous-programmes connus;
3. normaliser les candidats équivalents dans le domaine tout en préservant les alternatives non prouvées équivalentes;
4. vérifier le typage, les effets et les préconditions;
5. calculer les conséquences dans un domaine déclaré;
6. rechercher contradictions, contre-exemples et situations sous-déterminantes;
7. choisir une probe qui discrimine plusieurs candidats ou une batterie de cas adversariaux;
8. exécuter la probe, mettre à jour les preuves et conserver les résultats négatifs;
9. tester sur de nouvelles compositions et environnements;
10. promouvoir, garder comme candidat, réfuter ou marquer non résolu selon les critères.

10.3 Pluralité, sous-détermination et complexité

Un corpus fini peut être compatible avec de nombreuses fonctions. Toute identification dépend d'une classe d'hypothèses, d'une grammaire, de propriétés de simplicité, d'observations discriminantes ou d'hypothèses de domaine. Les principes de parcimonie servent à ordonner la recherche, non à transformer la simplicité en preuve de vérité. Si plusieurs modèles restent compatibles, ils sont conservés ou une observation plus informative est demandée.

10.4 Correction de modèles candidats

La validation d'un programme candidat sépare fidélité sur les exemples, respect des contraintes, performance sur holdout, comportement sous perturbation, qualité des explications, coûts, sécurité des effets et calibration. Un candidat peut être bon sur une mesure et échouer sur une autre. Le système garde un vecteur de mesures et une décision qui cite le profil d'usage visé.

11 Observation, reproduction et clonage comportemental

11.1 Représentation d'une démonstration

Une démonstration contient la cible autorisée, le protocole de capture, les entrées, contexte, états connus, actions, sorties, horodatages, événements et incertitudes. Les données d'entraînement et de test sont séparées par épisode, entité, environnement ou graine lorsqu'une proximité permettrait la mémorisation.

11.2 Quatre niveaux distincts

1. **Rejeu exact :** reproduire une trace connue. Le succès prouve seulement la reproduction du cas capturé.
2. **Imitation entrée-sortie :** retrouver les sorties attendues pour les entrées couvertes.
3. **Modèle de transition :** reconstruire les règles d'évolution d'un état sous une action dans un domaine observé.
4. **Programme/politique compositionnelle :** découvrir une procédure réutilisable dont les invariants se maintiennent dans des variantes et combinaisons inédites.

Ces niveaux ont des mesures et seuils indépendants. Une reproduction parfaite sur entraînement ne doit pas masquer la généralisation faible sur perturbations.

11.3 Boucle de clonage et reconstruction

La trace observée est convertie en données typées; plusieurs explications et algorithmes candidats sont produits; le système synthétise un clone de référence borné; il mesure les écarts; il génère des entrées nouvelles et des changements de contexte; il cherche des contre-exemples; il révise le modèle; il conserve l'original et la version candidate avec des manifestes et des rapports de comparaison. La cible cachée et les sorties de test ne sont pas injectées dans le clone. Si une interface d'observation, un format ou un domaine n'est pas encore pris en charge, une compétence correspondante est spécifiée et ajoutée, avec autorisation et protocole de contrôle.

12 Modèles de transition et monde simulé : usage conditionnel, testable et composable

12.1 Fonctions possibles, non mandat universel

Un modèle de transition peut prédire $\hat{s}_{t+1}, \hat{r}_t = \hat{T}(s_t, a_t, c_t)$, simuler des branches de plan, suggérer des probes ou économiser des interventions coûteuses. Il est représenté comme une composi-

tion de représentation d'état, modèle d'observation, transition conditionnée par action, prédiction d'incertitude, simulateur de branches, comparateur et validateur. Chaque sortie de ce sous-système est marquée PREDICTED ou SIMULATED; elle ne devient jamais une observation réelle par auto-cohérence.

Un modèle de transition n'est pas imposé à toute tâche. Un calcul pur ou une preuve symbolique avec sémantique complète peut ne pas l'utiliser; une tâche dynamique peut en tirer un bénéfice. La décision relève de la qualité mesurée de la décision finale, du coût total et du risque, pas du seul score prédictif.

12.2 Examen de la preuve historique O62/O63

L'archive FulleGenBrain_1 contient un dossier de revue qui cite les rapports d'expérience Order 62 et Order 63. Dans le rapport O62, la comparaison rapportée donne Base : 3 `PASS`, 4 `FAIL`, 11 `EXHAUSTED`; RS : 2 `PASS`, 4 `FAIL`, 12 `EXHAUSTED`; récompense moyenne rapportée 0,6833 pour Base contre 0,1972 pour RS. Le rapport indique que le résultat ne permet pas de promouvoir RS, dont l'effet est mixte et la moyenne appariée inférieure; la version finale le garde optionnel et désactivé par défaut. La campagne d'ablation courte est décrite comme non discriminante pour plusieurs composants, donc elle ne justifie pas de conclusions causales fortes sur chacun.

Le rapport O63 décrit 320 records, 1436 transitions, 1162 en entraînement, 274 en holdout et 27 actions opaques; il rapporte une MSE latente holdout de $1{,}7310954885 \times 10^{-4}$ et 27/27 tests logiciels. Ces signaux n'ont pas suffi à garantir une amélioration décisionnelle générale : dans ses résultats RS dédiés, symbolic est PASS à 1,0 dans les deux bras alors que routing est PASS (2,6) au baseline et EXHAUSTED (1,2) avec RS. Le rapport conclut que le scoreur apprend une structure mais que son bénéfice décisionnel général n'est pas établi et le désactive par défaut.

Ces résultats sont les constats rapportés par les artefacts historiques; le présent dossier n'a pas réexécuté l'intégralité de leurs campagnes. Ils ne démontrent pas l'inutilité générale d'un modèle du monde. Ils montrent plutôt qu'une MSE basse ou un scoreur bien ajusté ne suffisent pas à démontrer une amélioration des décisions et que l'activation doit être déterminée par ablation appariée.

12.3 Porte d'utilité empirique

Pour chaque famille de tâches, comparer sous mêmes données, splits, seeds, budget, vérificateur et nombre d'appels : (A) voie sans modèle de transition; (B) voie avec modèle réel; (C) contrôle nul/permuté conservant le coût d'appel; (D) variante ablatée (un-pas, incertitude ou imagination séparés). Un bras oracle peut servir de diagnostic seulement s'il ne fuite aucun label dans le bras déployable. Mesurer la réussite certifiée, l'utilité décisionnelle, les régressions par famille, la calibration, l'erreur sous perturbation, le nombre de probes réelles économisées, la latence et la mémoire.

La porte d'activation est préenregistrée :

$$\text{ActivateWM}(D) = \mathbf{1}\{\text{LCB}(U_B - U_A) > \delta_D \ \wedge \ Safety_B\},$$

où U est une utilité multi-critère explicitée, LCB une borne d'incertitude et δ_D le gain minimal utile défini avant l'évaluation. Le modèle n'est activé pour une famille que si le bénéfice répliqué dépasse le seuil et les portes de sûreté; sinon le contrôleur utilise une autre voie pour cette famille, sans détruire le modèle ni conclure qu'il ne pourra jamais être utile ailleurs.

12.4 Prédiction, reconstruction, intervention et explication

Une bonne prédiction observationnelle ne prouve pas la structure causale. La reconstruction d'une trajectoire, l'évaluation sous intervention, une explication contrefactuelle et une prédiction multi-pas sont des contrats distincts. Le test causal demande des interventions autorisées, un contrôle d'information accessible au modèle, une comparaison à des baselines et un vérificateur indépendant. En cas de sous-détermination, les modèles alternatifs sont conservés et une probe discriminante est recherchée.

12.5 Boucle de planification compositionnelle

La planification choisit des séquences de compositions sous contraintes de coût, risque et état attendu : observer l'état, générer des plans candidats, simuler seulement si le modèle a franchi sa porte d'utilité pour ce contexte, vérifier les préconditions, exécuter une action autorisée, observer le résultat réel, comparer prédiction et réalité, puis mettre à jour modèle et mémoire. La décision réelle est contrôlée après action; la simulation n'est pas assimilée à l'exécution. Le modèle peut être endormi lorsqu'il ne sert pas la tâche et réveillé si un routeur de capacité le demande, après contrôle de version et de domaine.

13 Langage et sémantique sans modèle de langage neuronal

13.1 Architecture du traitement linguistique

Le traitement linguistique s'appuie sur une chaîne de composants explicites et composables : normalisation; segmentation/tokenisation déterministe; grammaires et parseurs; analyse morphologique si nécessaire; arbre syntaxique; résolution des références; rôles sémantiques; portée de la négation/quantification/modalité; temps/aspect; contexte discursif; ambiguïtés concurrentes; conséquences testables; validation par contraintes et données. Les méthodes statistiques légères sont admises comme composants documentés, mais aucun modèle génératif de langage neuronal ou service de ce type ne fait partie de l'architecture.

13.2 Analyse concurrente

Pour un énoncé ambigu, le système produit plusieurs analyses structurées, chacune associée aux fragments qui l'étayent, aux hypothèses, aux contextes, aux conséquences et aux tests qui permettraient de la distinguer. La désambiguïsation peut reposer sur une grammaire, le contexte, la cohérence avec les faits, les contraintes ou une question de clarification. Le choix est révisable et garde sa provenance. L'absence d'analyse suffisante déclenche le développement de la grammaire ou du parseur manquant, pas une sortie inventée.

13.3 Compositionnalité de la sémantique

Le sens d'une expression est calculé à partir de la structure syntaxique, du sens des éléments et des règles de composition déclarées :

$$[\![f(e_1, \ldots, e_n)]\!]_c = \mathsf{ComposeMeaning}_f([\![e_1]\!]_c, \ldots, [\![e_n]\!]_c).$$

La fonction est éventuellement relationnelle lorsqu'il y a plusieurs sens compatibles. Les relations de coréférence, quantification, portée et discours ne sont pas réduites à des scores de similarité. Les interprétations produites restent des hypothèses jusqu'à validation selon leur tâche.

13.4 Réutilisation des idées multirésolutions

Les représentations hiérarchiques et vues multirésolutions peuvent être réutilisées si elles sont explicites, déterministes ou correctement documentées, traçables et testées pour leur tâche : chaîne brute, tokens, syntagmes, structure, références, sens, discours. On n'introduit pas un objectif prédictif de tokens en tant que dépendance neuronale du noyau; on mesure plutôt les performances de chaque représentation et ses effets sur la recherche, l'ambiguïté, le transfert compositionnel et le coût total.

14 Raisonnement formel et place d'eGenBrain v4

14.1 Responsabilité spécialisée

eGenBrain v4 reste un moteur spécialisé de maillage relationnel, d'inférence typée et d'application de règles/opérateurs déclarés. Il traite les prémisses, hyperrègles, contraintes de types et unités, clôtures définies et traces de dérivation dans le périmètre explicitement pris en charge. Il n'est pas responsable, à lui seul, de découvrir chaque règle absente, de sélectionner toutes les observations utiles ni de certifier la vérité des prémisses.

14.2 Intégration compositionnelle

La composition FulleGenBrain_3 appelle eGenBrain v4 par un adaptateur typé contenant : le contexte de requête, les prémisses admissibles, les règles/operators activés, la limite de fermeture, les contraintes d'unités, les versions, le budget et le schéma de certificat. La réponse est convertie en objet DERIVED avec ses dépendances. Le vérificateur indépendant contrôle les contraintes pertinentes et l'intégrité de la dérivation avant toute promotion à CERTIFIED.

14.3 Application, découverte et retrait

Si une règle explicite existe, l'engine peut l'appliquer selon sa sémantique. Si elle n'existe pas, la découverte appartient au moteur de candidats et de composition. Si une prémisse est retirée, les dépendants sont réévalués. Les caches de fermeture sont invalidés selon leur clé de prémisses, règles, contexte et version d'opérateur. Les conflits sont conservés avec leurs preuves plutôt que supprimés par dernier-écrit-gagne.

15 Mémoire, provenance et révision continue

15.1 Horizons de mémoire

La mémoire comprend : (A) épisodique – observations et essais; (B) de travail – contexte et MGR/MGR2 temporaires; (C) structurelle – faits, règles, programmes et compositions approuvés; (D) procédurale – méthodes et politiques validées; (E) de preuve – certificats, dépendances, révocations et tests; (F) de maintenance – incidents, checkpoints, versions et opérations à réconcilier. Ces catégories sont des vues logiques, pas nécessairement des bases physiquement séparées.

15.2 Cycle de révision

Chaque changement de preuve ou de contexte déclenche la recherche des dépendances affectées. L'objet concerné et tous les descendants reçoivent un statut périmé/retrait, puis sont réévalués selon les contrats. La revalidation peut construire une nouvelle preuve alternative; elle ne réactive pas silencieusement l'ancien certificat. Les historiques sont conservés comme événements passés, séparés de l'ensemble de prémisses actives.

15.3 Provenance complète

Tout objet issu d'une composition garde l'identifiant du programme, l'empreinte du DSL normalisé, la version du compilateur/runtime, les identifiants de blocs, les paramètres, les entrées pertinentes, les résultats de chaque étape, la configuration, le budget consommé et la preuve du vérificateur. La chaîne de hash assure la détection de modifications sur les artefacts couverts mais doit être combinée à l'authentification des acteurs si une garantie d'origine est requise.

16 Mémoire vivante : activation, compression et circulation d'information

16.1 Organisation hiérarchique et composable

La mémoire est composée de services typés plutôt que d'une seule table : stockage exact durable, index, cache chaud, cache tiède, archive froide, graphe de provenance, graphe de dépendances, magasin de checkpoints, index de compositions, moteur de réveil, révocation et files événementielles. Les niveaux sont des politiques de placement et d'encodage, non des niveaux de vérité. Toute donnée conserve son statut épistémique et ses pointeurs vers la source.

16.2 Objets conservés et types de compression

La mémoire peut conserver des données, observations, résumés, interprétations, règles, programmes, compositions, modèles de transition, états, expériences, certificats, réfutations, contre-exemples, coûts, erreurs, transformations, politiques, checkpoints et historique de versions. La factorisation de sous-graphes et la compression sans perte ne modifient pas le contenu canonique. Un résumé sémantique avec perte est une vue dérivée qui indique les distinctions préservées, le niveau d'incertitude et les usages permis; les requêtes demandant davantage de précision réveillent les sources exactes.

16.3 Veille/réveil avec intégrité et versions

Les compositions non utilisées sont candidates à la veille si elles ne sont ni épinglées, ni louées par un exécuteur, ni engagées dans une transaction, ni indispensables à une invalidation prioritaire. Le checkpoint persiste AST canonique, payload, hash, versions, dépendances, budgets, état de recherche, trace et état d'effets externes. Au réveil, la chaîne est : restaurer, vérifier l'empreinte, comparer les versions, contrôler contexte/permissions, revalider les contrats et enfin activer. Une composition dont les dépendances ont changé demeure STALE jusqu'à une décision explicite de revalidation.

Le score d'éviction dépend de l'utilité récente, du coût de maintien, du coût de réveil, de la prochaine utilisation probable, de la taille active et du risque de péremption. Il est appris ou calibré via des profils, puis évalué contre une politique de référence. Les données irremplaçables, certificats, journaux d'invalidation, checkpoints utiles à la reprise et preuves référencées par des objets actifs disposent de règles de conservation dédiées.

16.4 Vagues d'information comme flux d'événements

Une vague est un événement typé qui contient id, source/version, destinataire, catégorie, charge, contexte, priorité, TTL, compteur causal et provenance. Les vagues suivent les abonnements et dépendances, pas une diffusion non ciblée. Déduplication, limitation de fan-out, TTL, ordre stable, files persistantes et backpressure contrôlent la charge. La priorité la plus haute appartient aux révocations, changements de version et incidents de sûreté. Tout événement supprimé ou expiré laisse une trace de décision et le pointeur de sa source.

16.5 Contrats mesurables de la mémoire

Les métriques incluent empreinte mémoire active, taille froide, ratio réel de compression, coût des réveils, taux de réveil superflu, cache-hit, erreurs de fraîcheur, délai de propagation des invalidations, duplication d'événements, pertes de messages, backlog, latence et taux de régression après réveil. Aucun gain de compression n'est accepté au prix d'une perte silencieuse de preuve ou de provenance. Une campagne compare mémoire toujours active, veille seule, compression sans perte, résumé avec récupération et politique complète; les mêmes requêtes sont utilisées pour mesurer qualité et coûts.

17 Contrôleur, objectifs et allocation de ressources

17.1 Objectif comme objet formel de premier rang

Chaque tâche est encodée dans un objet versionné

$$G = (g, \mathcal{D}, \mathcal{C}, \mathcal{R}, \pi, h, \rho, \ell, \mathcal{A}, \sigma),$$

où g est la cible, $\mathcal{D}$ le domaine, $\mathcal{C}$ les contraintes, $\mathcal{R}$ les ressources, π la politique de priorité, h l'horizon, ρ le risque acceptable, ℓ le niveau de preuve requis, $\mathcal{A}$ les critères d'acceptation et σ la politique d'arrêt. Les préférences ordonnent le travail, mais ne neutralisent jamais une précondition, une restriction de capacité ou une porte de sûreté exigée par le contrat. Le synthétiseur doit chercher dans l'espace des compositions compatibles avec G, et tracer les raisons pour lesquelles un candidat est admis, écarté ou conservé comme non résolu.

17.2 Objectif vectoriel

Les mesures de tâche restent distinctes :

$$\begin{aligned}\mathbf{J} = (&L_{observation}, L_{induction}, L_{composition}, L_{imitation},\\ &L_{transition}, L_{semantique}, L_{contraintes}, L_{calibration},\\ &C_{calcul}, R_{effets}).\end{aligned}$$

Ce vecteur est une structure d'évaluation, pas une fonction d'entraînement unique. Le contrôleur applique un ordre de priorités explicite : exigences dures de validité et de sécurité; préconditions; contrat demandé; objectifs qualité; coût et préférences. Un score moyen ne peut pas compenser un certificat invalide ou une entrée de provenance corrompue.

17.3 Contrôle anytime et budget

Chaque requête définit ou hérite d'un budget pour étapes, temps, mémoire, probes, profondeur de composition, recherche de programmes, simulations et vérification. Le budget s'étend dynamiquement si une politique autorisée le permet, ou se répartit en fonction du gain attendu et des obligations encore ouvertes. À l'échéance, le système retourne le meilleur état justifié avec statut partiel, la liste des tâches restantes, le checkpoint et le plan de reprise. L'épuisement de budget n'est pas classé comme absence de capacité.

17.4 Sélection de voie

Le contrôleur choisit entre exécution d'une composition certifiée, recherche de composition nouvelle, observation supplémentaire, raisonnement formel, récupération mémoire, simulation, clonage, réparation du programme ou demande d'information. Le choix utilise la disponibilité et les preuves pertinentes, l'incertitude, la valeur des informations, le coût, les risques et la réversibilité. Lorsque deux voies indépendantes peuvent vérifier un résultat, le contrôleur peut demander une double vérification; la diversité est alors documentée.

18 Résilience opérationnelle et continuité

18.1 États de panne et actions

Les pannes sont typées : entrée invalide; bloc absent; contrat violé; dépendance/version incompatible; autorisation manquante; timeout; épuisement de ressource; composant indisponible; conflit; corruption; régression; résultat indéterminé; interruption du processus. Pour chaque type, le composant déclare retry, backoff, fallback compatible, checkpoint, compensation, rollback, réconciliation, continuation partielle ou arrêt contrôlé.

18.2 Sauvegarde, restauration et reprise

Le checkpoint comprend état sérialisé, version de schéma, version des compositions, journal des événements confirmés, sorties intermédiaires, graphes de dépendance, opérations externes en attente, budgets restants et hashes des artefacts. Au redémarrage, les schémas et manifestes sont validés. Les opérations d'effet externe dont l'issue est inconnue passent en réconciliation plutôt qu'être exécutées une seconde fois sans contrôle. Les écritures critiques sont atomiques ou protégées par un protocole de reprise explicite.

18.3 Tolérance au changement de composants

Si un bloc est absent ou déprécié, le catalogue cherche un substitut dont le contrat couvre l'usage. L'équivalence est une obligation à démontrer, pas un simple appariement de noms. Si le substitut n'offre qu'une partie des garanties, le contrôleur compose une stratégie alternative qui rétablit les garanties manquantes, par exemple en ajoutant une vérification ou une observation. Si aucune voie déjà disponible n'est adéquate, le bloc manquant devient un chantier de développement, priorisé et traçé.

18.4 Auto-réparation

L'auto-réparation génère un candidat de correction à partir du symptôme, du journal, de la composition et du test de régression. Elle reproduit le problème dans une sandbox, formule des hypothèses, construit un patch ou une transformation, exécute les tests négatifs et positifs, compare au comportement approuvé, puis soumet l'artefact au contrôle d'intégrité. Aucun patch ne modifie en place la version active sans checkpoint et voie de retour arrière.

19 Sécurité des effets sans limitation artificielle des tâches

La sécurité repose sur les capacités explicites, le principe du privilège minimal, la validation des chemins, le contrôle des identifiants de composants, la non-exécution de fragments arbitraires issus d'une composition, l'isolation et la traçabilité. Une tâche qui nécessite une capacité légitime additionnelle déclenche la définition d'une nouvelle interface et une autorisation appropriée; elle n'est ni exécutée par contournement silencieux ni abandonnée sans analyse. Le langage reste évolutif via extension revue, versionnée et testée.

Les effets sont classés comme lecture, écriture, réseau, exécution, observation interactive, horloge, aléatoire, commande externe ou modification persistante. Les contrats indiquent les droits, la portée, les compensations et les conditions d'activation. Le runtime refuse une composition mal formée ou insuffisamment autorisée avec un diagnostic précis; le système peut ensuite proposer une solution alternative sûre et complète, dans les limites réelles de ses capacités.

20 Preuve de composition et portes de vérification

20.1 Contrat de bout en bout

Pour une composition $p = c_n \circ \cdots \circ c_1$, la certification exige : contrats d'entrée/sortie composables; préconditions satisfaites; effets autorisés; versions connues; trace complète; invariants préservés; résultat vérifié par le contrôleur de qualité du domaine. Sous ces hypothèses, les règles de composition de la théorie compositionnelle propagent les garanties des blocs. Le certificat nomme les hypothèses et la portée; il ne prétend pas garantir les contextes non couverts.

20.2 Vérificateurs indépendants

Un vérificateur de syntaxe/compilation, un vérificateur de contrat, un vérificateur métier/domaine, un contrôleur de provenance et un évaluateur de généralisation ont des rôles distincts. Plusieurs

peuvent être regroupés dans une distribution logicielle mais leurs responsabilités et résultats restent séparés. Le composant qui cherche une solution ne doit pas décider seul qu'il a réussi.

20.3 Statuts de sortie

Les sorties admissibles sont : CERTIFIED; TESTED; SUPPORTED; PARTIAL; UNRESOLVED; REFUTED; FAILED; NEEDS-OBSERVATION; NEEDS-COMPETENCE. Les deux derniers lancent soit une nouvelle expérience, soit un travail explicite de construction et intégration de la capacité nécessaire. Aucun statut n'est remplacé par du langage rassurant qui dépasse les preuves.

21 Intégration complète de la théorie compositionnelle dans FulleGenBrain_3

La théorie compositionnelle est le langage commun des capacités, des stratégies, de la mémoire et des procédures de validation. Cette intégration est exhaustive au niveau de la spécification : chaque constructeur est relié à sa sémantique, à une responsabilité d'architecture, à l'état réel du prototype et aux tests/obligations correspondants. Une fonction spécifiée mais non présente dans le prototype demeure visible comme travail d'implémentation; la matrice interdit d'inférer sa disponibilité à partir de sa seule présence dans le DSL.

Opérateur théorique	Rôle et contrat dans FulleGenBrain_3	Couverture actuelle du noyau de référence
Id	Identité pure avec trace conforme au contrat.	Sémantique spécifiée; interpréteur DSL non fourni dans ce prototype.
Call	Résolution versionnée d'un composant, contrôle type/contrat/capacité.	Registre de compositions présent; invocation DSL générique non intégrée.
Seq	Enchaînement avec validation des pré/postconditions et propagation des statuts.	Spécifié; exécuteur AST de séquence à construire.
Par	Exécution concurrente seulement sous contrats d'effets compatibles.	Spécifié; runtime parallèle et vérification d'indépendance à construire.
If	Branchement sur garde typée, trace du garde et branche exécutée.	Spécifié; nœud AST non intégré au runtime livré.
Map	Application bornée à chaque élément avec politique d'erreur.	Spécifié; map du DSL FGB3 non intégré au runtime livré.
Fold	Réduction avec accumulateur typé, invariant et ordre défini.	Spécifié; moteur de réduction à construire.
Loop	Itération avec invariant, budget et preuve de terminaison par rang si revendiquée.	Spécifié; boucle DSL non intégrée au runtime livré.
Choose	Maintien d'alternatives sans choix arbitraire implicite.	Spécifié; gestion d'alternatives dans le synthétiseur à construire.
Select	Choix via une politique versionnée parmi les candidats admissibles.	Best-first générique trie un front; opérateur AST/selecteur sémantique non intégré.
Verify	Vérification indépendante et résultats VALID/INVALID/UNKNOWN.	Porte d'activation du modèle de transition testée; vérificateur générique de programme non intégré.

Opérateur théorique	Rôle et contrat dans FulleGenBrain_3	Couverture actuelle du noyau de référence
Observe	Capture contrôlée d'observations avec source, contexte et provenance.	Contrat spécifié; collecteurs DDD intégrés non livrés dans ce runtime.
Probe	Choix d'une observation discriminante selon gain, coût et risque.	Contrat spécifié; planificateur de probes non intégré.
Recover	Récupération bornée, compensation et statut partiel tracé.	Non intégré comme opérateur DSL; restauration mémoire locale testée séparément.
Txn	Transaction locale ou protocole de compensation explicite.	Non intégré; la transaction distribuée reste une obligation d'implémentation.
Checkpoint	Point de cohérence du programme, de la recherche et de l'état.	Checkpoint du front de recherche testé; snapshot général d'exécution non intégré.
Revoke	Révocation et propagation vers les descendants dépendants.	Invalidation sur changement de version partiellement implémentée/testée; opérateur DSL absent.
Compose	Instanciation d'un sous-programme paramétré avec contrats et versions.	Non intégré comme nœud exécutable.
Transform	Transformation candidate avec diff, preuve/évaluation, approbation et rollback.	Non intégré; mécanisme d'auto-transformation à construire et évaluer.
Fix	Récursion/point fixe avec sémantique et conditions de terminaison.	Spécifié; non intégré au runtime livré.
Synthesize	Génération de candidats sous objectif, grammaire, contraintes et budget.	Front best-first reprenable disponible; moteur de génération AST/domaines non intégré.
Hibernate	Mise en veille après checkpoint, sous protections de bail/pin/transaction.	Implémenté dans `LivingCompositionMemory`; tests positifs/négatifs inclus.
Wake	Réveil avec vérification hash/version, fraîcheur et revalidation.	Implémenté via `activate/lease`; versions périmées et corruption testées.
Compress	Compression sans perte du payload canonique.	zlib/SQLite et mesures de ratio implémentées pour le payload mémoire, non en opérateur DSL général.
Summarize	Vue avec perte et contrat de fidélité, référence exacte conservée.	Spécifié; résumé sémantique à perte non implémenté.
Restore	Restauration avec intégrité, compatibilité de schéma et réconciliation.	Restauration payload/checkpoint de recherche partiellement implémentée; effet externe non inclus.
EmitWave	Émission persistante d'événements typés avec déduplication et backpressure.	File durable et émission implémentées/testées dans le prototype.
ReceiveWave	Validation, livraison et traitement idempotent d'une vague.	File/livraison/forwarding testés; handlers métier génériques non intégrés.

Opérateur théorique	Rôle et contrat dans FulleGenBrain_3	Couverture actuelle du noyau de référence
ResumeSearch	Reprise d'un front versionné sans perdre les signatures/compteurs.	Checkpoint/restore de `ResumableBestFirstSearch` implémenté/testé.

21.1 Hiérarchie des abstractions apprises

Le catalogue distingue *Primitive*, *Operator*, *Skill*, *Method*, *Strategy*, *Policy* et *MetaStrategy*. Une primitive expose une opération élémentaire; un opérateur règle la composition; une compétence est un contrat réutilisable orienté tâche; une méthode compose des compétences; une stratégie choisit entre méthodes; une politique associe contexte et action/méthode; une méta-stratégie ajuste le choix des stratégies sous une procédure contrôlée. La promotion est une transformation versionnée qui exige des tests de variation, une portée de validité et une provenance; un résultat dans des exemples ne suffit pas à universaliser une compétence.

21.2 Différences structurantes avec FulleGenBrain_2

La V3 conserve la composition comme substrat central et ajoute : (i) sémantique contractuelle explicite des opérations de mémoire/événements; (ii) gestion intégrée du cycle de vie actif/veille/réveil/stale; (iii) mémoire chaude/tiède/froide avec compression exacte et résumés qualifiés séparément; (iv) propagation événementielle bornée et traçable; (v) défense multicouche contre l'explosion combinatoire; (vi) porte empirique pour activer les modèles de transition par famille; (vii) matrice entre tous les constructeurs théoriques et leurs statuts d'implémentation. Ces ajouts précisent l'architecture sans prétendre que le prototype implémente déjà tout le runtime cible.

22 Protocole d'évaluation de FulleGenBrain_3

22.1 Suite de tests à construire

1. Composition pure : identité, séquence, associativité, normalisation et égalité fonctionnelle.
2. Composition à effets : ordre d'écriture, conflit de ressources, idempotence, rollback et compensation.
3. Compilation : validateur AST, interpréteur de référence, compilateur, exécution différée et diff des sorties.
4. Synthèse : règle cachée, programme à exemples, grammaire étendue, ambiguïté, contre-exemples et compositions inédites.
5. DDD et probes : partitions d'observations, coût d'information, stabilité des vues et changement de contexte.
6. MG/MGR/MGR2/ASCR : promotion, provenance, révocation, fermeture, conflits, cache et reprise.
7. Clonage : rejeu, imitation, transitions, perturbations et transfert hors des démonstrations.
8. Sémantique : négation, portée, quantificateurs, coréférence, références, temps, paraphrases et contexte multi-tour.
9. Résilience : crash injecté, checkpoint corrompu, composant absent, version incompatible, timeout et opérations externes ambiguës.
10. Auto-transformation : changement de composant, refactorisation, régression, évaluation OOD, approbation et rollback.

22.2 Métriques

Les mesures comprennent : correction du résultat; taux de violations de contrat; validité des certificats; taux de fausses certifications; taux de réfutation correcte; couverture de recherche; diversité des candidats; gain informationnel par probe; nombre d'observations; fidélité de reproduction par niveau; généralisation sur compositions inédites; exactitude sémantique; calibration/abstention; coût CPU/mémoire/latence; consommation des probes; taux de reprise; temps de restauration; régressions induites et temps d'ingénierie nécessaire à l'ajout d'une capacité.

22.3 Comparaisons causales et ablations

Les essais comparent : runtime seul; catalogue sans synthèse; synthèse sans probes; DDD sans recherche active; maillage sans graphe de provenance; eGenBrain v4 seul dans son domaine; composition complète; composition sans clonage; sans modèle de transition; sans revalidation; avec contrôles de secours indépendants. Les comparaisons gardent les données, budgets, splits et oracles constants. Les ablations identifient contributions et interactions, sans préjuger du signe ou de la taille de l'effet.

22.4 Prévention des fuites et reproductibilité

Les jeux sont partitionnés par composant, entité, environnement, template, épisode ou famille de composition, selon la menace de fuite. Les oracles sont séparés du runtime; les cas cachés ne sont pas accessibles au synthétiseur. Les résultats publient versions, graines, budgets, erreurs, tests non lancés, dépendances, hashes et conditions d'environnement. Les résultats répétés ne sont pas additionnés sans indiquer les répétitions.

23 Plan d'intégration recommandé

Phase	Travail principal	Porte de sortie
P0	contrats universels, schémas, statuts, registre et provenance	schémas validés et tests de sérialisation
P1	DSL/AST, interpréteur de référence, runtime séquentiel et branchements	tests sémantiques et différentiel
P2	vérificateur indépendant, effets, capacités, manifests et intégrité	tests négatifs et contrats
P3	MG/MGR/MGR2/ASCR, révocation et dépendances	fermeture/invalidation et reprise contrôlées
P4	synthèse de règles et de compositions sur tâches structurées	holdout, contre-exemples et métriques
P5	observation DDD et sondes discriminantes	valeur des probes et absence de fuite
P6	modèle du monde, trajectoires et clonage	perturbations et généralisation
P7	analyse sémantique composable	suite linguistique structurée
P8	résilience, checkpoint, compensation et autoréparation	panne injectée et rollback
P9	intégration complète et ablations	rapports de tâches end-to-end

Ce plan ordonne les dépendances de construction sans figer les priorités scientifiques. Une tâche critique rencontrée en avance est formalisée puis branchée à la couche dont elle dépend; les portes restent nécessaires avant de déclarer une garantie.

24 Correspondance avec FulleGenBrain_1, AAM O94.6 et eGenBrain v4

Élément	Apport retenu	Modification pour FulleGenBrain_3
FulleGenBrain_1	contrat universel, statuts épistémiques, DDD, maillage, clonage, probes, modèles du monde et vérification	composition comme langage commun; mémoire vivante par veille/réveil; contrôle multi-couche de la recherche; activation conditionnelle des modèles de transition; aucune dépendance de langage neuronal
AAM O94.6	catalogue de blocs, composition déclarative, compilation, traces, manifests, build/test et démonstrateurs bornés	étendre le socle aux contrats formels, effets, programme de synthèse, transaction, révocation, reprise et transformation testée
eGenBrain v4	représentation relationnelle, inférence typée et application des règles déclarées dans son périmètre	l'intégrer comme moteur spécialisé invoqué par des compositions, sans lui attribuer toute induction
Order	recherche de trajectoires, exploration et organisation des essais	formaliser les heuristiques comme stratégies composables et mesurables, jamais comme preuves
AAM (hors seule composition)	observation, réutilisation de blocs, corrections, traces et pratiques de qualité issues des artefacts disponibles	chaque capacité est réécrite en contrat et validée indépendamment de la marque/source historique
DDD	empreintes, partitions et distinctions dépendantes des probes	relier explicitement les changements de mesure aux hypothèses, à la sélection d'expériences et à la provenance
Idées d'eLLM	multi-vues, hiérarchies, évaluation des séquences	réimplémentation explicite; aucun appel, poids ou dépendance LLM/eLLM

25 Questions ouvertes comme tâches de construction

Les items ci-dessous sont des chantiers concrets qui doivent être fermés par formalisation, implementation et tests; ils ne sont pas des restrictions définitives de l'architecture : preuve machine de l'équivalence interpréteur/compilateur; logique de types/effets plus expressive; sémantique de la concurrence et de l'annulation; modèles de transactions externes; apprentissage automatique de nouveaux opérateurs de programme; induction de représentations plus robustes; sémantique large du langage; observabilité d'environnements hétérogènes; évaluation à long terme de l'auto-transformation; garanties d'équité, de coût et de stabilité. Chaque item reçoit un identifiant, un propriétaire de composant, des critères de sortie, des dépendances, un banc d'essai et un journal des preuves.

26 Conclusion et statut

FulleGenBrain_3 est organisé autour d'une thèse d'ingénierie : les capacités cognitives doivent être représentées, reliées et transformées comme des programmes compositionnels typés, révisables et

vérifiables. Le reste de l'architecture – observation, DDD, maillage, induction, clonage, modèles du monde, raisonnement formel, sémantique, mémoire et autonomie – devient une collection de compétences et de contrôleurs aux contrats explicites. Cette organisation permet de raisonner sur les dépendances, le réemploi, les régressions et la continuité sans confondre les statuts de preuve.

Le présent livrable est une spécification formelle architecturale produite après examen ciblé des sources disponibles et du démonstrateur de programmation compositionnelle AAM. Il ne prétend pas certifier l'intégration de tous les composants ni une performance globale. Les preuves de ce document sont déductives sous leurs hypothèses; les résultats des sources et les tests exécutés sont cités à leur portée exacte. La prochaine étape scientifique est de fermer une verticale intégrée : objectif, recherche compositionnelle, veille/réveil, observation, test, réfutation, révision, certificat et mémoire; puis d'évaluer séparément la contribution du modèle de transition par ablations appariées. Le noyau de référence livré avec ce dossier vérifie localement des mécanismes de cycle de vie mémoire, propagation d'événements et recherche reprenable; il ne constitue pas une implémentation complète de FulleGenBrain_3.